From Exact Anchors to Simultaneous-Move AlphaZero for the STL Handkerchief Game

Repository whitepaper

July 10, 2026

Abstract

This whitepaper records both the implemented solver and the revised plan for reinforcement learning. The game is modeled as a fully observed, two-player, zero-sum stochastic game with simultaneous exact-second actions. The Python engine is the rule oracle. The rigorous solver performs finite-horizon backward induction with engine-derived chance and linear-programming minimax at every simultaneous node. The approximate stack provides candidate search, matrix-game Monte Carlo tree search (MCTS), tablebase frontiers, and a policy/value network.

The repository does not yet implement a complete AlphaZero loop. Phases P0–P3 of the Python bridge now provide a finite episode contract, exact/search conformance, reconstructable replay and checkpoints, and a tested simultaneous-MCTS improvement object; a short Generation-Zero pipeline smoke also passes. Full anchor generation is in progress and full training remains unexecuted. The compatibility self-play module still uses a legacy bucketed agent against scripted opponents. This distinction matters: AlphaZero-style reinforcement learning requires search policies at states actually visited by MCTS self-play and terminal episode outcomes as value targets. Rust is deferred until the Python behavior and evidence package are stable. We prove the finite-horizon exact result, the non-expansiveness of minimax backup, a frontier-error bound, and a local saddle gap bound. We also state precisely why these results do not prove convergence or global exploitability of the learned agent.

1 Scope, Status, and Claims

The live runtime tree is under `stl/`. The disconnected old package mirror was removed after pushed archival commit `a4b03db`; Git, rather than a second source tree, retains that history. The implementation plan is `docs/REGEN2RL.md`, and the granular removal ledger is `docs/DISTILLATION_AUDIT.md`. This paper supplies the mathematical rationale and claim boundary.

Path	Responsibility
`stl/engine/`	Rule authority, clock, roles, legality, death, revival, and leap behavior
`stl/solver/exact.py`	Exact public state, engine expansion, LP minimax, and finite-horizon solve
`stl/solver/search.py`	Candidate search, bounded resolve, leaf adapters, and matrix-game MCTS
`stl/solver/tablebase.py`	Tactical fixtures, certified intervals, and Tier A lookup
`stl/learning/`	Targets, policy/value model, training, replay support, and gates
`stl/play/`	Playable agents, scripted opponents, environment wrappers, and legacy bucket baseline
`stl/commands/`	Hydra command implementations
`configs/`	Resolved experiment and command parameters

The following claims are justified now:

- engine-relative correctness of exact action legality and transition expansion, subject to the regression suite;
- exact minimax value for the finite-horizon game actually enumerated by the full-width LP solver;
- exact values of pinned analytic fixtures by their construction; and
- finite-depth best-response intervals under their reported frontier brackets.

The following claims are not justified now:

- that the unrestricted-duration game has been solved;
- that finite-budget candidate MCTS converges under the present selection rule;
- that held-out mean squared error implies low exploitability;
- that existing self-play code is AlphaZero self-play; or
- that a Rust implementation is required for, or already defines, the learning baseline.

2 Game Model

2.1 Fully Observed State with Simultaneous Actions

Let the players be $\mathcal{N} = \{\text{Hal}, \text{Baku}\}$. At a nonterminal public state $s \in \mathcal{S}$, both players observe the complete state represented by `ExactPublicState`: player cylinders, total time dead, death counts, life status, physicality, referee CPR count, clock, half-round, round number, first dropper, and terminal fields. The players then choose their current actions without observing the other current action. This is a perfect-state or fully observed simultaneous-move game, not an alternating-action board game and not a hidden-state poker game.

The engine assigns a dropper $D(s)$ and checker $C(s)$. It is convenient to write the two player action sets as $A_{\text{Hal}}(s)$ and $A_{\text{Baku}}(s)$, then map the resulting pair to a drop second d and a check second c according to the roles. The code stores matrices as drop-by-check and transposes or negates them so that Hal remains the maximizing player.

2.2 Actor-Aware Exact-Second Legality

Normal legal seconds are $1, \dots, 60$. In the leap window only Baku, when Baku is the dropper, may select second 61. Checkers are always capped at 60, and Hal can never select 61. Thus

$$A_D(s) = \begin{cases} \{1, \dots, 61\}, & D(s) = \text{Baku and } s \text{ is a leap turn}, \\ \{1, \dots, 60\}, & \text{otherwise}, \end{cases} \qquad A_C(s) = \{1, \dots, 60\}.$$

Dense learned policies have length 62. Index 0 is illegal padding and index j denotes literal second j.

The check succeeds iff $c \geq d$. On success the checker accumulates $c - d$ seconds, including zero on the diagonal. On failure the checker incurs the fixed 60-second penalty. All death, revival, clock, and leap effects then come from `stl/engine/game.py`.

2.3 Transition Kernel and Utility

Let

$$P(s' \mid s, a_{\text{Hal}}, a_{\text{Baku}})$$

be the transition kernel induced by the engine, including survival and fatal branches. The rigorous solver does not learn or reimplement this kernel.

Terminal utility is always Hal-perspective:

$$u(s) = \begin{cases} +1, & s \text{ is terminal and Hal wins,} \\ -1, & s \text{ is terminal and Baku wins,} \\ 0, & s \text{ is an explicitly declared draw.} \end{cases}$$

There is no intermediate shaped reward in `stl/solver`.

2.4 Why Reinforcement Learning Needs an Episode Contract

Observation 1 (The engine does not force a finite episode)**.** *A repeated diagonal action pair $c = d$ succeeds with squandered time zero. Such a path need not increase either cylinder and can therefore remain nonterminal for arbitrarily many half-rounds.*

Consequently, a completed-game outcome is not guaranteed merely by calling the engine in a loop. The first Python RL baseline defines a capped training game G_H: after H half-rounds, a nonterminal trajectory is assigned training result zero and marked as truncated. The initial plan freezes $H_{\text{train}} = 64$ and $H_{\text{eval}} = 128$, then gates promotion on low truncation rate and sensitivity to the cap. This is an RL-wrapper adjudication, not a new engine terminal rule. Any result learned under this contract is a result about G_H, not automatically about the unrestricted-duration game.

3 Exact Finite-Horizon Solver

Assumption 1 (Engine authority)**.** *The Python engine in `stl/engine` is the rule oracle. Solver correctness is relative to its legality and transition functions.*

Let $b : \mathcal{S} \to [-1, 1]$ be a boundary evaluator. For a terminal state, the boundary is ignored in favor of u. Define

$$V_0^b(s) = \begin{cases} u(s), & s \text{ terminal,} \\ b(s), & s \text{ nonterminal,} \end{cases}$$

and, for $h > 0$, define the Hal-payoff matrix

$$M_h^b(s)_{ij} = \sum_{s'} P(s' \mid s, a_{\text{Hal},i}, a_{\text{Baku},j}) V_{h-1}^b(s').$$

The Bellman-Shapley backup is

$$V_h^b(s) = \text{val}\Big(M_h^b(s)\Big) = \max_{\sigma \in \Delta(A_{\text{Hal}}(s))} \min_{\tau \in \Delta(A_{\text{Baku}}(s))} \sigma^{\mathsf{T}} M_h^b(s) \tau.$$

For a matrix $M \in \mathbb{R}^{m \times n}$, the row-player LP is

$$\begin{aligned} \max_{\sigma, v} \quad & v \\ \text{subject to} \quad & \sum_{i=1}^{m} \sigma_i M_{ij} \geq v, \quad j = 1, \ldots, n, \\ & \sum_{i=1}^{m} \sigma_i = 1, \\ & \sigma_i \geq 0, \qquad\qquad i = 1, \ldots, m. \end{aligned}$$

The certified code path uses full legal exact-second actions and this LP. With the current terminal-only exact cutoff, $b(s) = 0$ and unresolved probability is tracked separately.

Proposition 1 (Exact finite-horizon value)**.** *For fixed engine state s, finite horizon h, full legal action sets, and boundary evaluator b, recursive engine expansion followed by exact LP minimax returns $V_h^b(s)$.*

Proof. The claim is immediate for $h = 0$. Assume it holds at $h - 1$. For each legal joint action at s, the engine enumerates all successor chance branches and the induction hypothesis gives the continuation value at every successor. Their probability-weighted sum is therefore the corresponding entry of $M_h^b(s)$. The one-step decision is a finite zero-sum matrix game, so von Neumann's minimax theorem equates its LP value with $\mathrm{val}(M_h^b(s))$. This is $V_h^b(s)$, completing the induction. □

4 Error Reasoning for Depth-Limited Search

4.1 Minimax Backup Is Non-Expansive

Lemma 1 (Matrix-value Lipschitz bound)**.** *If two equally shaped payoff matrices satisfy $\|M - \widehat{M}\|_\infty \leq \epsilon$, then*

$$\left|\mathrm{val}(M) - \mathrm{val}(\widehat{M})\right| \leq \epsilon.$$

Proof. For every pair of mixed strategies (σ, τ),

$$\left|\sigma^{\mathsf{T}}(M - \widehat{M})\tau\right| \leq \epsilon$$

because σ and τ are probability vectors. Taking a minimum and then a maximum cannot enlarge this uniform bound. Applying the same argument with M and $\widehat{M}$ exchanged gives the absolute-value claim. □

Proposition 2 (Frontier error does not amplify under exact backup)**.** *Suppose two boundary evaluators satisfy $\sup_s |b(s) - \widehat{b}(s)| \leq \epsilon$. If all actions and transitions are expanded exactly, then for every finite horizon h,*

$$\sup_s |V_h^b(s) - V_h^{\widehat{b}}(s)| \leq \epsilon.$$

Proof. At depth zero the result is the hypothesis. If it holds at depth $h - 1$, then every action-cell continuation differs by at most ϵ, since an expectation is non-expansive in the supremum norm. The matrix-value lemma then bounds the backed-up value difference by ϵ. Induction proves the claim. □

This proposition justifies calibrated frontier values for an exact depth-limited resolve. It does not say that a finite-sample MCTS estimate has a uniform error bound, nor that a compact feature map can represent such a bound.

4.2 Local Policy Quality from Matrix Accuracy

For a Hal row strategy $\widehat{\sigma}$ and Baku column strategy $\widehat{\tau}$, define the exact matrix saddle gap

$$\mathrm{gap}_M(\widehat{\sigma}, \widehat{\tau}) = \max_{\sigma} \sigma^{\mathsf{T}} M \widehat{\tau} - \min_{\tau} \widehat{\sigma}^{\mathsf{T}} M \tau.$$

It is zero at an exact saddle point and measures the total one-step gain available from unilateral deviations.

Proposition 3 (Approximate-matrix saddle gap)**.** *Let $(\widehat{\sigma}, \widehat{\tau})$ be an exact equilibrium of $\widehat{M}$. If $\|M - \widehat{M}\|_\infty \le \epsilon$, then*

$$\mathrm{gap}_M(\widehat{\sigma}, \widehat{\tau}) \le 2\epsilon.$$

Proof. Let $\widehat{v} = \mathrm{val}(\widehat{M})$. Uniform matrix error implies

$$\max_{\sigma} \sigma^{\mathsf{T}} M \widehat{\tau} \le \widehat{v} + \epsilon, \qquad \min_{\tau} \widehat{\sigma}^{\mathsf{T}} M \tau \ge \widehat{v} - \epsilon.$$

Subtracting the second inequality from the first gives the result. □

This provides a concrete policy gate: whenever full-width exact matrices are available, report saddle gap in addition to value error.

4.3 Candidate Sets Need Their Own Audit

No analogous bound follows merely because a candidate set contains many actions or tactically plausible seconds. A restricted matrix can be identically zero while one omitted Hal action pays +1 against every Baku action. The restricted value is then zero and the full value is one. Candidate search is therefore an approximation whose omitted-action best-response gain must be measured against full width on a frozen audit distribution.

5 Current Search Stack

5.1 Selective Solve and Bounded Resolve

`stl/solver/search.py` generates candidate seconds around endpoints, diagonals, safe budgets, and overflow boundaries. Its selective recursion still uses engine transitions and role-correct matrix minimax, but its action restriction is approximate. Critical-state resolve applies a hard horizon, usually one half-round, and evaluates the frontier with a tablebase or network adapter. LP remains the certification path; Python CFR+ is a runtime option for bounded local matrices.

5.2 Matrix-Game MCTS

Each MCTS node stores candidate role actions, a Hal-perspective mean Q-matrix, joint cell counts, priors, children, and an exact engine snapshot. The current exploration term is

$$U_{ij} = c_{\mathrm{puct}} P_{ij} \frac{\sqrt{N}}{1 + N_{ij}}.$$

The Hal maximizer solves an optimistic matrix $Q + U$; the opponent solves an optimistic matrix for its own payoff, equivalently $-Q + U$. Joint actions are sampled from the two resulting role

marginals, chance is sampled through the engine, and Hal-perspective leaf values are backed up without alternating sign.

Let $\widehat{\sigma}_D^{(t)}$ and $\widehat{\sigma}_C^{(t)}$ be role equilibria of the root's empirical mean-Q matrix before the t-th root selection. The canonical improvement policy is their linearly weighted average

$$\pi_D = \frac{1}{W_T}\sum_{t=1}^{T} t\widehat{\sigma}_D^{(t)}, \qquad \pi_C = \frac{1}{W_T}\sum_{t=1}^{T} t\widehat{\sigma}_C^{(t)}, \qquad W_T = \sum_{t=1}^{T} t.$$

Later empirical matrices receive greater weight without collapsing mixed play. The optimism-augmented equilibria select exploratory joint cells but are not reported as policy probability. Network priors are masked and normalized per role, their outer product defines the joint prior, and optional root Dirichlet noise perturbs the two legal marginals independently. The final saddle point of the mean Q-matrix remains a separate value/policy diagnostic; in a nearly flat matrix an LP may return an arbitrary pure vertex, so it is not the canonical training or deployed policy.

The convergence theorem for simultaneous-move MCTS in [4] assumes Hannan-consistent selection, guaranteed exploration, and a finite game tree. The repository's PUCT-like matrix selection, neural frontier, candidate actions, transpositions, and finite episode wrapper are not automatically covered by that theorem. The frozen exact-matrix convergence pack therefore supplies empirical, not theorem-level, evidence: across 120 fixture/budget/seed records its 1024-iteration median absolute value error is zero, 95th-percentile error is 0.001812, maximum saddle gap is 0.005726, maximum per-fixture root-value standard deviation is 0.000736, and no fixture worsens from 256 to 1024 iterations. A future regret-matching SM-MCTS replacement remains appropriate if broader frozen packs violate the declared gate.

6 Audit of the Current Learning Attempt

6.1 Implemented Pieces

The repository already contains several components needed by the target system:

- `stl/learning/model.py`: a shared trunk, Hal value head, and two length-62 role-policy heads driven by 52 named V2 features;
- `stl/learning/contracts.py`: the versioned finite-episode, perspective, configuration, and horizon-sensitivity contract;
- `stl/learning/replay.py`: reconstructable `TrainingRecordV2` rows, safe hash-verified shards, collision audits, and grouped splits;
- `stl/learning/train.py`: value MSE, masked role-policy cross entropy, source weights, grouped selection, and strict checkpoint bundles;
- `stl/learning/targets.py`: exact/tablebase labels and MCTS bootstrap labels;
- `stl/learning/support/reanalysis.py`: deeper exact attempts and a high-iteration MCTS fallback;
- calibration, audit, ladder, policy-drift, finite-depth best-response, SPRT, and promotion utilities; and

- `stl/play/agent.py`: a playable network-plus-MCTS agent that samples the canonical improved role policies.

The P1 and P3 frozen conformance packs test exact transition/orientation parity, omitted-action gains, chance sampling, and horizon-one MCTS convergence. The P4 smoke writes and reloads small Generation-Zero V2 train and external ruler shards. These are valuable prerequisites; they do not yet form one closed RL loop.

6.2 Blocking Integration Gaps

- **No premature command surface.** The broken legacy `command=self_play` wrapper has been removed. A public command is added only when P5 supplies the joint MCTS actor, reconstructable replay, and command-contract tests.
- **Legacy actor.** `stl/learning/support/self_play.py` uses bucketed `CanonicalHal` against scripted opponents. It is curriculum or legacy outcome data, not MCTS self-play by both sides.
- **Expert-iteration labels.** MCTS bootstrap sweeps a constructed state grid and labels root value; it does not supply trajectory outcomes from the improved policy.
- **Generation Zero is not frozen.** The structural smoke is complete, but the full exact/tablebase anchor corpus, external-ruler review, training run, and search-in-the-loop gate have not run.
- **Incomplete promotion evidence.** Policy drift is optional in promotion, and finite-depth best response is called certified exploitability; the first omits a policy gate and the second can be misread as a global claim.

The correct response is not to discard the exact stack. It is to connect the existing pieces with explicit schemas and gates, while quarantining legacy bucket search from the new self-play actor.

7 Target Python AlphaZero System

7.1 Reconstructable State and Replay

Each `TrainingRecordV2` row carries a versioned 52-field feature vector and a serialized exact public state. It also records both legal masks, both improved role policies, Hal-perspective value target, target kind, episode and half-round indices, truncation status, parent checkpoint digest, search/config digest, schema versions, and RNG seeds. Pickle-free numeric shards and a canonical JSON manifest are written atomically and verified by hashes. This turns replay into an auditable experiment record rather than an opaque tensor dump.

The V2 encoder includes every nonconstant value-relevant field in `ExactPublicState` and appends documented relative features. A compact network remains an approximation, but replay reconstruction permits collision analysis and exact or MCTS reanalysis. Split assignment is grouped by exact-state hash and episode so repeated or replicated states cannot cross train and validation boundaries.

7.2 Policy/Value Function

The network is

$$f_\theta(x(s)) = (p_D(\cdot \mid s), p_C(\cdot \mid s), v_\theta(s)),$$

where each policy is masked and normalized over the actor-aware legal seconds for its role, and $v_\theta(s) \in [-1, 1]$ estimates Hal outcome. Role heads are appropriate because the state encoder identifies which named player holds each role.

The two policies define independent mixed actions. A joint correlated policy would define a different strategic object, because it could coordinate the players' simultaneous actions. Therefore the joint prior and sampling law are factorized:

$$P_{ij} = p_D(i \mid s)p_C(j \mid s), \qquad d \sim \pi_D(\cdot \mid s), \quad c \sim \pi_C(\cdot \mid s).$$

7.3 MCTS Self-Play

At every visited state, one search produces both improved role marginals. Self-play exploration perturbs the legal marginals separately and records the noise seed. Evaluation disables noise. Because a simultaneous equilibrium may require mixing, evaluation samples the noise-free marginals rather than taking an argmax.

The actor stores temporary rows

$$(s_t, \pi_{D,t}, \pi_{C,t}, m_{D,t}, m_{C,t})$$

and samples the next engine transition. At the end of the episode, it attaches $z \in \{-1, 0, +1\}$ to every row, with zero reserved for the explicit capped draw contract when no engine draw exists. Root MCTS value is logged for diagnosis but is not mislabeled as the terminal outcome.

7.4 Learning Objective

For replay sample (s, π_D, π_C, z), the AlphaZero-style loss adapted to two role heads is

$$\mathcal{L}(\theta) = \lambda_v (z - v_\theta(s))^2 - \lambda_D \sum_{a \in A_D(s)} \pi_D(a \mid s) \log p_D(a \mid s) - \lambda_C \sum_{a \in A_C(s)} \pi_C(a \mid s) \log p_C(a \mid s) + c\|\theta\|_2^2.$$

This is the direct analogue of the AlphaZero objective [7], with separate simultaneous role policies. Exact and tablebase anchors may substitute certified value and equilibrium-policy targets. Rows without a certified policy have that policy loss disabled; they do not receive a fabricated uniform label.

The initial replay mixture keeps 25 percent immutable exact/tablebase anchors and 75 percent self-play rows. This ratio is an engineering baseline, not a convergence theorem. Its purpose is to prevent catastrophic drift from exact anchors while the empirical promotion gates judge whether self-play improves strength.

7.5 Reanalysis

Reanalysis reconstructs stored states and refreshes the search policies using the current frozen champion. Completed episode outcomes remain unchanged. Exact and tablebase rows are immutable. Search-bootstrap values, if retained for experiments, carry a distinct target-kind tag so they cannot be confused with Monte Carlo outcomes.

8 Promotion and Evidence

Promotion is more conservative than AlphaZero's policy of continually using the latest network. AlphaGo Zero's evaluate-and-select lineage, together with the repository's observed calibration and strength regressions, motivates a frozen champion/candidate gate.

Every candidate must pass:

1. artifact and configuration digest consistency;
2. exact/tablebase value calibration and role-policy legality;
3. the exact-matrix MCTS convergence pack;
4. a paired two-direction arena that respects Hal/Baku asymmetry and uses a predeclared SPRT;
5. non-regression against the scripted robustness league, including `pattern_reader`;
6. finite-depth best-response interval comparison at identical scenarios, horizons, support mass, and frontiers; and
7. a mandatory policy-drift/readability report with exact-anchor saddle gaps.

Best-response intervals certify only their bounded calculation. If $[L_C, U_C]$ and $[L_B, U_B]$ overlap, the comparison is inconclusive. A promotion report may say that no sampled case is certified worse; it may not infer global improvement or global low exploitability.

9 Phased Implementation and Rust Boundary

The executable plan and numeric exit criteria live in `docs/REGEN2RL.md`. Its dependency chain is:

1. freeze the objective, perspective, and finite episode contract;
2. certify engine/exact/search conformance in Python;
3. introduce reconstructable replay and checkpoint schemas;
4. validate matrix-game MCTS as the improvement operator;
5. train a supervised exact-anchor generation zero;
6. build genuine MCTS self-play for both roles;
7. close replay, training, and reanalysis;
8. integrate statistical promotion; and
9. freeze a reproducible Python baseline and evidence package.

Items 1–4 (P0–P3) are implemented and verified. A short item-5 pipeline smoke generated 26 reconstructable training rows and 26 external-ruler rows and reloaded a one-epoch checkpoint. The full item-5 anchor generation is the current stopping boundary; neither that long job nor full training has run.

No phase in this chain requires new Rust work. Existing Rust CFR+ bindings are opt-in comparison fixtures only. After the Python baseline is frozen, narrow profiled kernels may be ported behind parity tests. Python remains the oracle for legal actions, transitions, policy normalization, replay semantics, and promotion. The Rust acceptance ordering is behavioral parity first and speed second.

10 What the Proofs Do and Do Not Establish

The exact induction proves a finite-horizon result when all legal actions and chance branches are represented and the LP is solved. The non-expansive backup and saddle-gap propositions explain how a genuine uniform frontier or matrix error bound would control local value and policy error.

They do not provide such a uniform error bound for a neural network. Empirical MSE is distribution-dependent; feature collisions can force incompatible states to share a prediction. Finite-budget MCTS adds sampling and selection error. Candidate sets add unmeasured action error unless full-width audits are performed. Function approximation, replay nonstationarity, and policy/value co-adaptation remove the assumptions of tabular exact policy iteration.

Accordingly, the planned system is a rigorously anchored empirical solver, not a theorem of global AlphaZero convergence. A future global claim would require at least a well-defined unrestricted objective, exhaustive or otherwise valid full-game bounds, and a global exploitability certificate. Until then, reports must keep these categories separate:

- **exact**: full-width finite-horizon LP or analytic tablebase;
- **bounded**: interval or error statement for a declared horizon and frontier;
- **approximate**: finite-budget candidate MCTS or neural output;
- **empirical**: self-play, arena, calibration, and robustness results on declared distributions.

11 Validation Protocol

The minimum documentation-aligned regression commands are:

```
uv run pytest tests/engine -q
uv run pytest tests/solver -q
uv run pytest tests/learning -q
uv run pytest tests/play -q
uv run pytest -q
uv run pytest --collect-only -q
```

The stable-baseline gate additionally requires deterministic self-play replay, exact-state reconstruction, grouped split isolation, exact-matrix MCTS budget curves, a successful and a rejected promotion dry-run, horizon sensitivity, and a graph refresh by running `graphify update` against the repository root; generated checkpoints and reports remain outside source review unless the artifact itself is under audit.

12 Conclusion

The repository has a credible exact/search foundation: engine authority, literal-second stochastic minimax, tablebase anchors, bounded resolve, a tested matrix-game MCTS improvement object, reconstructable replay/checkpoints, and a policy/value learner. The remaining gap is a frozen Generation Zero, genuine MCTS self-play trajectories with terminal outcomes, a closed replay/reanalysis loop, and mandatory statistical promotion. Only after those Python phases produce a stable evidence package is the baseline suitable as an oracle for a later Rust kernel.

References

[1] John von Neumann. Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen*, 100:295–320, 1928. https://doi.org/10.1007/BF01448847

[2] Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953. https://doi.org/10.1073/pnas.39.10.1095

[3] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. *Advances in Neural Information Processing Systems*, 2007. https://papers.nips.cc/paper_files/paper/2007/hash/08d98638c6fcd194a4b1e6992063e944-Abstract.html

[4] Viliam Lisy, Vojtech Kovarik, Marc Lanctot, and Branislav Bosansky. Convergence of Monte Carlo tree search in simultaneous move games. arXiv:1310.8613, 2013. https://arxiv.org/abs/1310.8613

[5] Matej Moravcik, Martin Schmid, Neil Burch, Viliam Lisy, Dustin Morrill, Nolan Bard, Trevor Davis, Kevin Waugh, Michael Johanson, and Michael Bowling. DeepStack: Expert-level artificial intelligence in heads-up no-limit poker. *Science*, 356(6337):508–513, 2017. https://doi.org/10.1126/science.aam6960

[6] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550:354–359, 2017. https://doi.org/10.1038/nature24270

[7] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv:1712.01815, 2017. https://arxiv.org/abs/1712.01815

[8] Noam Brown, Anton Bakhtin, Adam Lerer, and Qucheng Gong. Combining deep reinforcement learning and search for imperfect-information games. *Advances in Neural Information Processing Systems*, 2020. https://proceedings.neurips.cc/paper/2020/hash/c61f571dbd2fb949d3fe5ae1608dd48b-Abstract.html